
Основы машинного обучения

Вопросы к контрольной работе

правки в tg @Pypsenish

April 07, 2025

Содержание

1. Что такое объекты и признаки в машинном обучении? Для чего нужен функционал ошибки? Что такое алгоритм (модель)?	3
2. В чём заключается обобщающая способность алгоритма машинного обучения? К чему приводит её отсутствие? Что такое переобучение? Что такое отложенная выборка? Что такое кросс-валидация (скользящий контроль)? Как ими пользоваться для выбора гиперпараметров?	3
3. Что такое гиперпараметр? Чем гиперпараметры отличаются от обычных параметров алгоритмов? Приведите примеры параметров и гиперпараметров в линейных моделях.	5
4. Как метод k ближайших соседей определяет класс для нового объекта? Опишите метод k ближайших соседей с парзеновским окном. Какие в нём есть параметры? Запишите формулу метода kNN для регрессии.	6
5. Что такое градиент? Какое его свойство используется в машинном обучении?	6
6. Опишите алгоритм градиентного спуска.	6
7. Что такое регуляризация? Как она помогает бороться с переобучением? Как подбирать коэффициент и в чем различия?	8
8. Что такое масштабирование признаков? Как его проводить? Зачем это нужно?	9
9. Чем функционал MSE отличается от MAE? Что такое функция потерь Хубера и для чего она нужна?	10
10. Как выглядит модель линейной классификации в случае двух классов? ...	11
11. Что такое отступ? Для чего он нужен?	12
12. Как обучаются линейные классификаторы (общая схема с верхними оценками)?	12
13. Что такое метод опорных векторов? Опишите его основную идею. На что влияет гиперпараметр C?	13
14. Чем отличаются функции потерь в логистической регрессии и в SVM?	15
15. Как можно свести задачу многоклассовой классификации к серии задач бинарной классификации?	16
16. Что такое микро- и макро-усреднение?	16

А ничо тот факт что контрольная и есть причина тряски?

На основе лекций Евгения Соколова на майноре ИАД и «хендбука Яндекса»

1. Что такое объекты и признаки в машинном обучении? Для чего нужен функционал ошибки? Что такое алгоритм (модель)?

Функционал ошибки крайне важная вещь, фактически именно через него мы сообщаем модели чего вообще хотим от неё и что хотим сообщить.

2. В чём заключается обобщающая способность алгоритма машинного обучения? К чему приводит её отсутствие? Что такое переобучение? Что такое отложенная выборка? Что такое кросс-валидация (скользящий контроль)? Как ими пользоваться для выбора гиперпараметров?

В контексте машинного обучения под «переобучением» понимается ситуация, когда модель фактически просто заучила обучающую выборку. В таком случае её качество на трейне будет близко к идеальному, однако на новых данных случится катастрофа, т.к. модель лишь «поняла алгоритм» решения задачи на обучающей выборке, но суть задачи осталась неясной.

С другой стороны мы можем выделить **обобщающую способность** – т.е. способности модели адаптироваться к новым данным и выдавать на них качество/ошибку не сильно хуже, чем при обучении

Как же различать эти две ситуации?

Ну давайте просто отрежем от нашей выборки некоторый кусочек, на котором в дальнейшем, после непосредственно обучения, будем просто тестировать нашу модель. И вот если она на тестовых данных себя хорошо показала, то модель действительно обобщилась, ура.

Отсюда и появляется **отложенная выборка** - то есть часть данных, которые модель не видит в процессе обучения, а лишь проверяет качество.

Кросс-валидация – алгоритм проверки качества (обобщающей способности) модели. Такой способ проверки обобщающей способности надежнее, чем обычное разбиение на train и test, но ощутимо медленнее.

Опишем алгоритм оценки качества:

- (i) Фиксируем некоторое число k , обычно от 3 до 10
- (ii) Наша выборка делится на k одинаковых по размеру частей, которые именуются фолдами
- (iii) Далее одна из этих k частей отправляется в test выборку, а объединение остальных – в train, на которой и происходит обучение модели. И так происходит ровно k итераций, где каждый раз часть, которая становится тестовой, меняется и не повторяется. То есть на каждом шаге модель учится на $k - 1$ фолде.

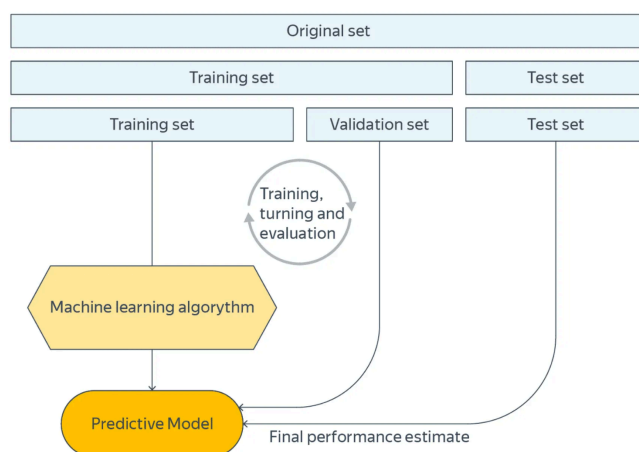
- (iv) Финальная оценка модели (ошибка) измеряется либо усреднением ранее получившихся результатов, либо можно заранее запастись ещё одним небольших набором данных, который до этого никак нигде не участвовал и уже измерить окончательно на нём.
- (v) Остаётся понять, какую из обученных моделей использовать, ведь на выходе у нас k различных экземпляров одной модели. Насколько я понял, можно выделить много подходов, опишу два:
 - (i) просто выбрать ту модель, у которой ошибка на своей тестовой части меньше
 - (ii) Т.к. часто кросс-валидация используется для сравнения нескольких подходов/методов к обучению модели, то можно таким образом сравнивать как раз несколько, выбрать лучшую, а затем обучить заново итоговую модель, но уже на всех k фолдах.

Также стоит отметить, что есть различные вариации кросс-валидации, такие как leave-one-out.

Такое большое разнообразие в делении данных требует как можно более эффективного data-менеджмента :)

А как будем подбирать при этом **гиперпараметры**? Здесь можно выделить несколько подходов, рассмотрим:

- (i) Просто делим датасет на 3 выборки: обучающую, валидационную и тестовую. На обучающей – обучаем, на тестовой – исключительно тестируем полностью готовую, а вот на валидационной крутим вертим гиперпараметры, минимизируя (максимизируя) ошибку (метрики качества) на этой выборке. Подбор гиперпараметров не стоит делать на тестовой выборке, всё-таки тестовая выборка предназначена исключительно для проверки обобщающей способности модели на абсолютно новых данных, которая она никогда ранее не видела. В случае подбора по тесту можно неявно заложить в модели информацию о тестовом множестве и получить результаты хуже ожидаемых на новых данных.



- (ii) Сделать **кросс-валидацию**, но предварительно отделить тестовую выборку от датасета и не трогать до самого конца. А на каждой итерации кросс-валидации обучать на $k - 1$ фолде, а последний использовать как валидационную выборку.

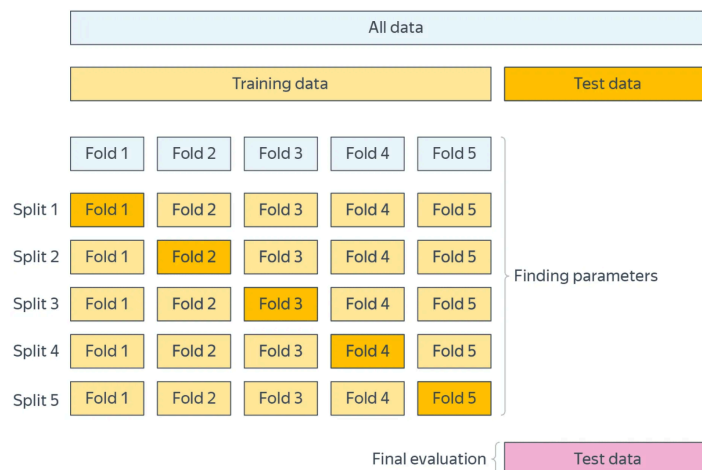


Рис. 2. Источник – **хендбук**

- (iii) Использовать более продвинутые оптимизационные алгоритмы, например, Grid или Random Search, о них можно почитать в хендбуке.

3. Что такое гиперпараметр? Чем гиперпараметры отличаются от обычных параметров алгоритмов? Приведите примеры параметров и гиперпараметров в линейных моделях.

Будем различать параметры и гиперпараметры:

- **параметры** – величины модели, которые непосредственно настраиваются и оптимизируются в процессе обучения модели.
 - В модели линейной регрессии $\frac{1}{l} \|Xw - y\|^2 + \lambda \|w\| \rightarrow \min_w$ это непосредственно вектор весов w , потому что именно вектор весов мы можем как-то менять в процессе обучения.
- **параметры** – определённые характеристики модели, которые настраиваются до обучения, но непосредственно влияют на то, какой модель получится по итогу.
 - Примеров можно привести довольно много: от k в knn до λ в регуляризации, η в градиентном спуске или C в SVM.

4. Как метод k ближайших соседей определяет класс для нового объекта? Опишите метод k ближайших соседей с парзеновским окном. Какие в нём есть параметры? Запишите формулу метода kNN для регрессии.

Скажи мне, кто твой
друг, и я скажу, кто ты
– Метод k-ближайший
соседей

Зачем как-то обучаться, зачем какие-то там SGD, опорные вектора или энтропии, когда можно просто посмотреть на соседние объекты и сказать, что ты такой же, ведь все мы +- одинаковые?

5. Что такое градиент? Какое его свойство используется в машинном обучении?

Из курса матанализа знаем, что градиент обозначает вектор частных производных некоторой многомерной функции. Важнейшее его свойство - функция быстрее всего растёт в направлении градиента этой функции в некоторой точке. Можно доказать (с помощью разложения в ряд Тейлора), что наискорейшее убывание будет в сторону антиградиента в точке.

Definition 5.1. (Градиент): Градиентом функции $f : R^d \rightarrow R$ называется вектор её частных производных: $\nabla f(x_1, \dots, x_n) = \left(\frac{\partial f}{\partial x_i} \right)_{i=1}^d$

6. Опишите алгоритм градиентного спуска.

Идея достаточно проста. Пусть у нас изначально есть какой-то набор параметров w , по которым мы оптимизируем некоторый функционал. Тогда этот набор мы можем улучшить, посчитав градиент функционала и сдвинувшись в противоположную сторону, обновив при этом исходный вектор неизвестных весов, т.е.

- имеем начальный набор весов (например для линрега) w_0
- Повторяем до сходимости следующее:

$$\underbrace{w_{n+1}}_{\text{новый вектор параметров}} = \underbrace{w_n}_{\text{старый}} - \underbrace{\eta_n \nabla Q(w_n)}_{\text{изменение}}$$

- где $Q(w_n)$ означает значение функционала в точке
- через η_n обозначим длину нашего «шага». Главное при выборе этой длины не сделать её слишком большой (тогда начнём просто проскакивать минимум) или слишком маленькой (тогда будем ждать сходимости долго). Можно делать монотонно убывающей, например, как $\eta_n = \frac{1}{n}$

Останавливать всю эту крутую процедуру можно при близости градиента к нулю, т.е. $\|\nabla Q(w_{n-1})\| < \varepsilon$, или при чересчур малом изменении вектора весов, т.е. $\|w_n - w_{n-1}\| < \varepsilon$, или при малом изменении самого функционала ошибки, т.е. $\|Q(X; w_n) - Q(X; w_{n-1})\| < \varepsilon$. Конечно, у этого оптимизационного алгоритма есть свои проблемы, в том числе большая вычислительная сложность и риск оказаться в локальном минимуме.

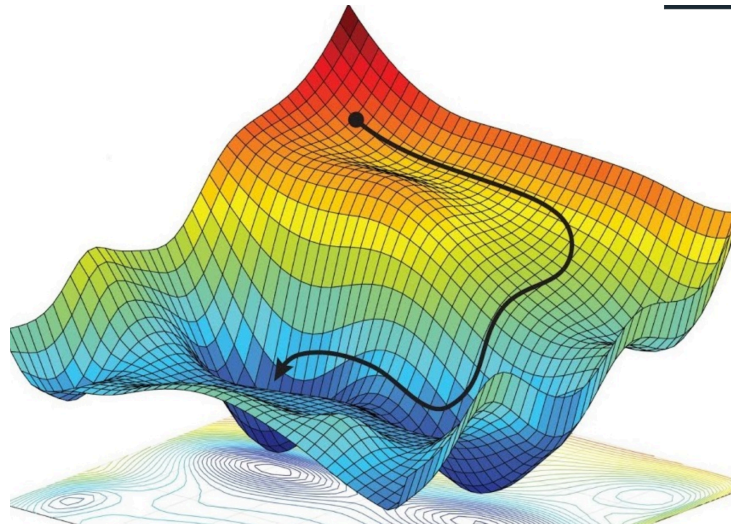
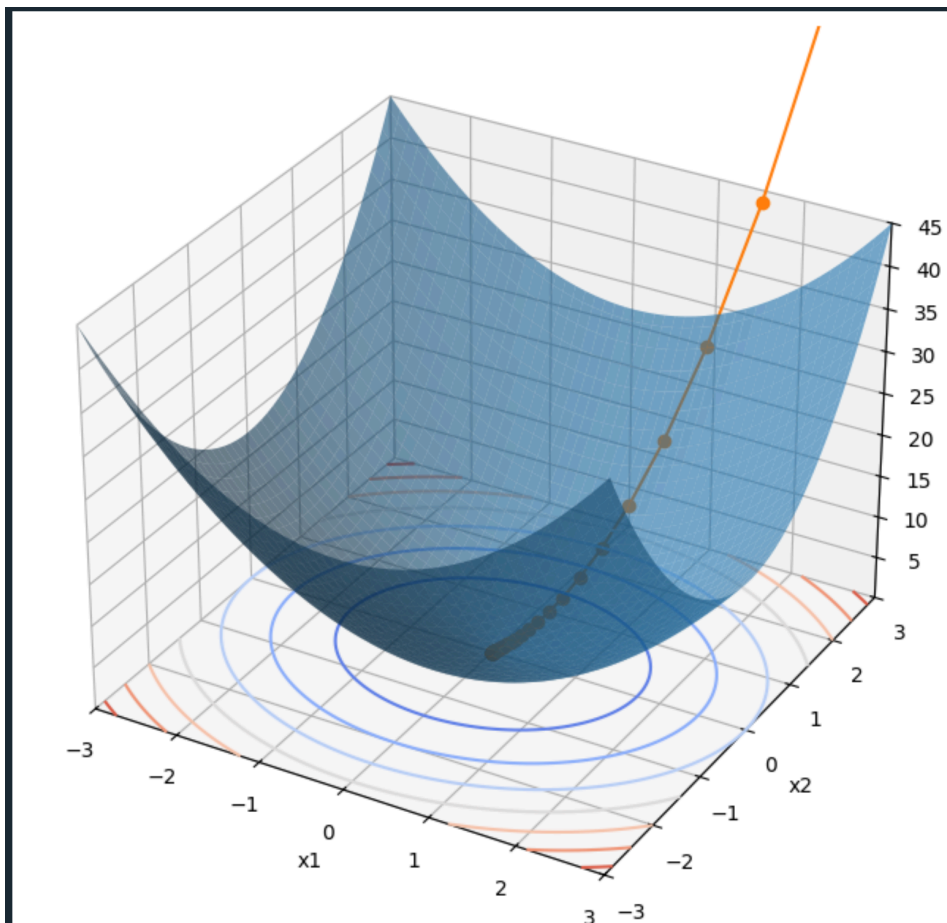


Рис. 3. Визуализация градиентного спуска



7. Что такое регуляризация? Как она помогает бороться с переобучением? Как подбирать коэффициент и в чем различия?

Одним из основных симптомов переобучения линейных моделей будут выступать большие веса. Вследствие этого теряется как физический смысл этих весов, так и интерпретируемость самой модели. В итоге это приводит к тому, что малейшие изменения в весах ведут к совершенно радикальным изменениям в прогнозе, но так не бывает, изменения должны быть «жизненно» объяснимыми и плавными.

Тогда будем запрещать модели иметь большие веса и дополнительно штрафовать за это, т.е. обновленный функционал ошибки будет иметь вид

$$\underbrace{\frac{1}{l} \sum_{i=1}^l (< w; x_i > -y_i)^2}_{\text{не ошибаться}} + \underbrace{\lambda \|w\|^k}_{\text{не переобучаться}} \rightarrow \min_w$$

λ - коэффициент регуляризации, гиперпараметр, насколько сильно запрещаем иметь большие веса.

Там регуляризация вроде способствует 100% существованию решения, т.к. матрица уже не будет вырожденной, но искать эту обратную матрицу это всё еще самоубийство при большой выборке.

Два основных регуляризатора:

- lasso $\sum_{i=1}^d |w_i|$
- через обычную евклидову норму в квадрате $\sum_{i=1}^d w_i^2$

L2 - регуляризация работает прекрасно и используется в большинстве случаев, но есть одна полезная особенность L1-регуляризации: её применение приводит к тому, что у признаков, которые не оказывают большого влияния на ответ, вес в результате оптимизации получается равным 0.

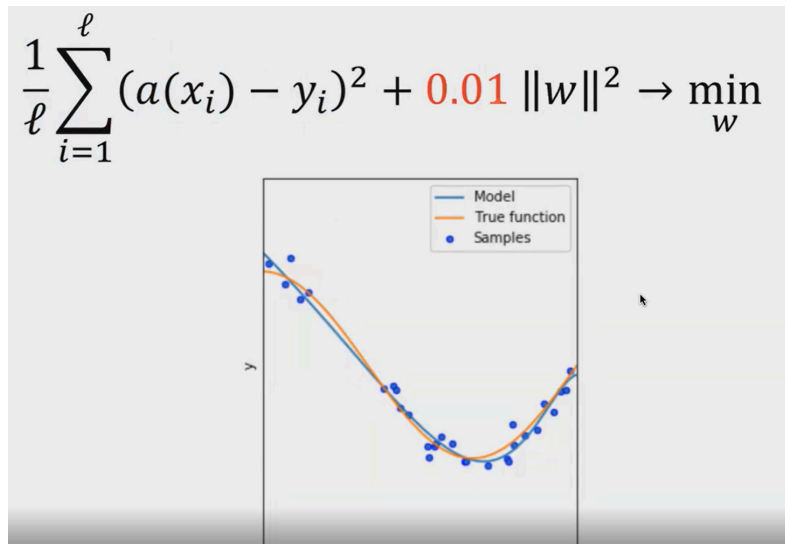


Рис. 4. Пример действия L2 регуляризации

Как будем подбирать коэффициент регуляризации? Ну для начала можно выбрать некоторую сетку из предполагаемых значений, далее обучим модель при каждом значении и посчитаем ошибку (можно просто на отложенной выборке, можно кросс-валидацией), выберем ту, где ошибка меньше всего с соответствующим значением регуляризатора.

Важно помнить, что в регуляризатор мы не включаем свободный коэффициент в векторе весов, т.к. фактически он отвечает за сдвиг нашей линейной прямой относительно начала координат, а целевую переменную мы как правило не масштабируем.

8. Что такое масштабирование признаков? Как его проводить? Зачем это нужно?

Под интерпретируемостью модели мы понимаем ситуацию, когда осознаем, как именно признаки будут влиять на прогноз, т.е. как вход модели влияет на выход. Вероятно, неплохо будет смотреть на вес этого признака в модели, однако в случае, если наши признаки изначально имеют разный масштаб вес перестаёт иметь смысл, т.к. в зависимости от масштаба мы будем получать и сильно разные значения веса.

Исходя из этого, хорошим тоном будет заниматься масштабированием признаков. Ну например через приведение среднего к нулю и стандартного отклонения к 1, т.е. выполнения такой процедуры

$$\frac{x_{ij} - \mu_j}{\sigma_j} \forall i \in \{1, \dots, \ell\}, \forall j \in \{1, \dots, d\}$$

Вроде после проведения масштабирования градиентный спуск сходится обычно быстрее.

9. Чем функционал MSE отличается от MAE? Что такое функция потерь Хубера и для чего она нужна?

Вспомним, как задается функционал mse:

$$Q(a, X) = \frac{1}{l} \sum_{i=1}^l (a(x_i) - y_i)^2$$

. Что с ней не так? - Ну вообще она крайне сильно чувствительна к выбросам, и для MSE важно уменьшить ошибку на выбросе больше, чем на норм объектах, т.е. если мы ухудшим прогноз на норм объектах, но улучшим на выбросе, то MSE может снизиться - это кринж.

Что же даёт нам функционал MAE? Это просто:

$$Q(a, X) = \frac{1}{l} \sum_{i=1}^l |a(x_i) - y_i|$$

и здесь уже не такой сильной чувствительности к выбросам. Возможно, строит обратить внимание на то, что из-за постоянных абсолютных значений производной у MAE градиентный спуск может проскакивать минимум, но я верю в то, что современные оптимизационные методы это могут легко фиксировать. У MSE же значение производной убывает по мере приближения к минимуму, поэтому обучаться на неё проще.

А теперь возьмем жесткое комбо из двух и получим вот такую конфетку под названием Хубер-лосс, которая выглядит как то так:

$$L_{H(X,a)} = \begin{cases} \frac{1}{2}(y-a)^2 & \text{if } |y-a| < \delta \\ \delta(|y-a| - \frac{1}{2}\delta) & \text{if } |y-a| \geq \delta \end{cases}$$

И обучается мощно и к выбросам норм, т.к. вырождается в MAE на выбросах

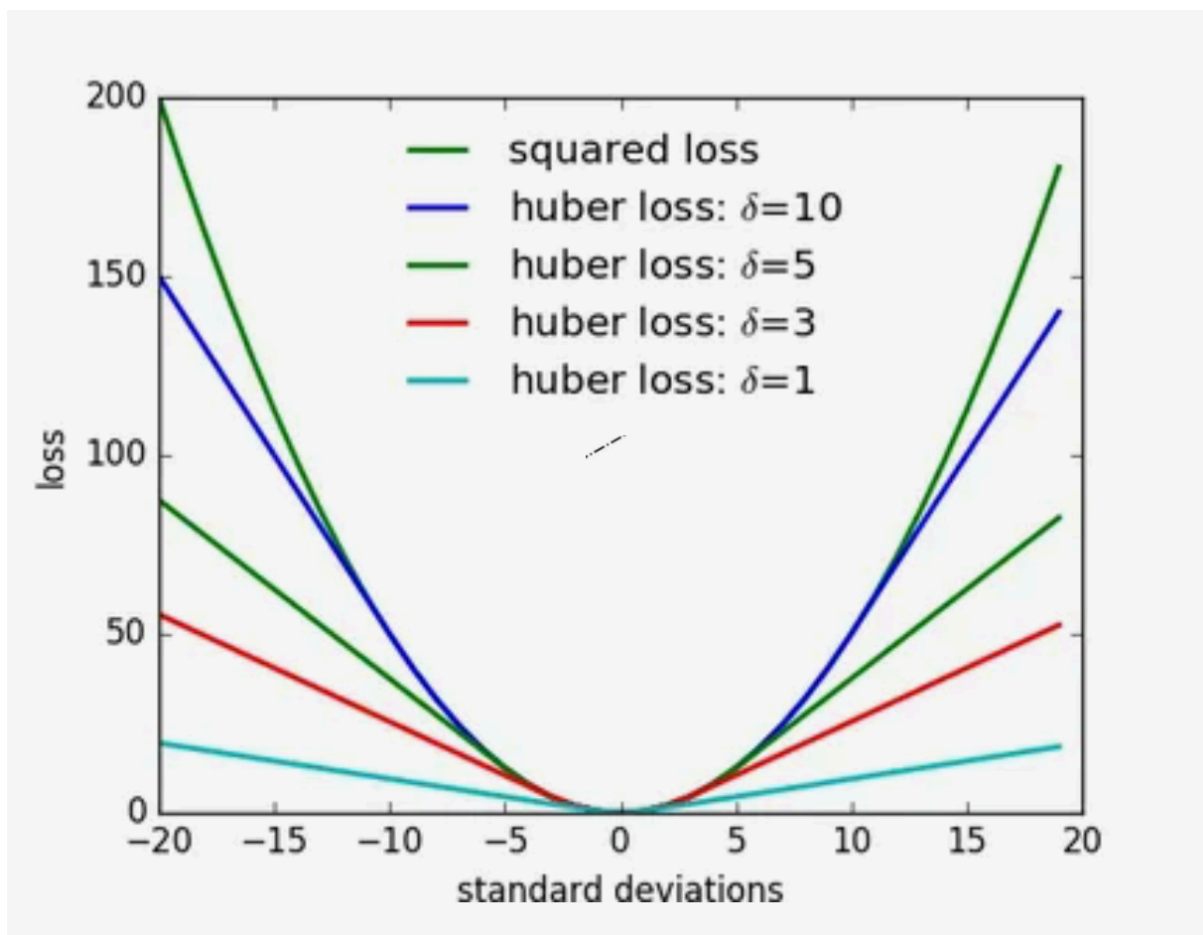
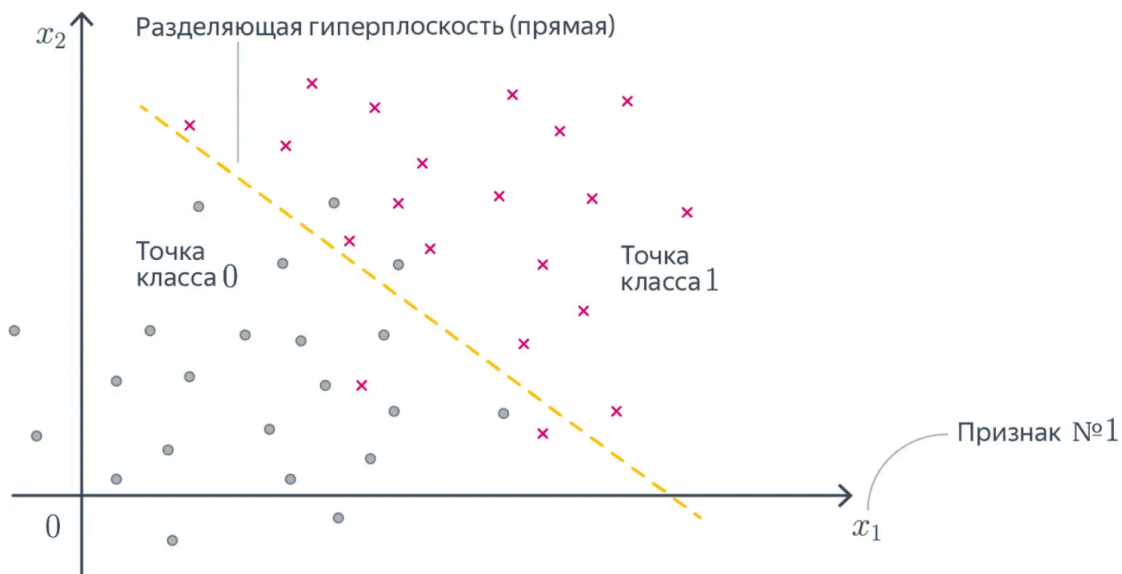


Рис. 5. Поведение Хубер-лосса при различных δ

10. Как выглядит модель линейной классификации в случае двух классов?

Перейдём к задаче бинарной классификации. То есть пространство ответов в этом случае $\mathcal{Y} = \{+1; -1\}$, т.е. модель должна возвращать одно из двух чисел $\rightarrow$ обычная регрессия не подойдёт, т.к. например даже если мы не угадаем верный класс, но при этом объект будет близко находится к разделяющей поверхности, то штраф будет небольшим, такое не подойдёт нам. Будем брать знак, т.е. $a(x) = \text{sign}\langle w; x \rangle$. Скалярное произведение при равенстве нулю будет давать нам гиперплоскость, которая будет разделять наши два класса, короче как на картинке. Хотим обучить модельку так, чтобы это разделение было как можно лучше, при этом вектор весов в данном случае является вектором нормали, а те объекты, которые будут давать ноль ($\langle w; x \rangle = 0$) будут находиться на самой разделяющей поверхности.



11. Что такое отступ? Для чего он нужен?

Под отступом будем понимать: $M = y_i < w; x_i >$

Отступ позволяет получить две важные характеристики:

- корректность ответа классификатора (в этом случае отступ всегда больше нуля)
- насколько модель уверена в ответе (об этом будет говорить величина отступа, т.к. $< w; x_i >$ на самом деле пропорционально расстоянию от объекта до разделяющей гиперплоскости, т.к. это расстояние вычисляется с помощью $\rho = \frac{\|< w; x_i >\|}{\|w\|}$). Поэтому чем ближе отступ к нулю, тем ближе объект к разделяющей поверхности, тем меньше модель уверена в прогнозе.

12. Как обучаются линейные классификаторы (общая схема с верхними оценками)?

Мы не сможем для классификации просто решать задачу регрессии в целом как минимум по двум причинам:

- Даже если объект будет находиться близко к разделяющей поверхности, но при этом мы не угадали класс, то ошибка будет минимальной, ведь в регрессии мы штрафует именно за то, насколько далеко наш прогноз оказался по сравнению с истинной
- Кроме этого, мы будем считать ошибкой если наша модель выдала, например, 5 в качестве ответа вместо 1, так как для нас пространство ответов ограничено $\{+1; -1\}$. Но это странно: число то положительное, поэтому класс мы определим правильно.

Поэтому логично потребовать просто как можно меньшее количество ошибок (привыкли что-то минимизировать, поэтому запишем задачу минимизации кол-ва неверных ответов, а не максимизации акураси):

$$Q(a, X) = \frac{1}{l} \sum_{i=1}^l [a(x_i) \neq y_i] \rightarrow \min_w$$

Перепишем в виде

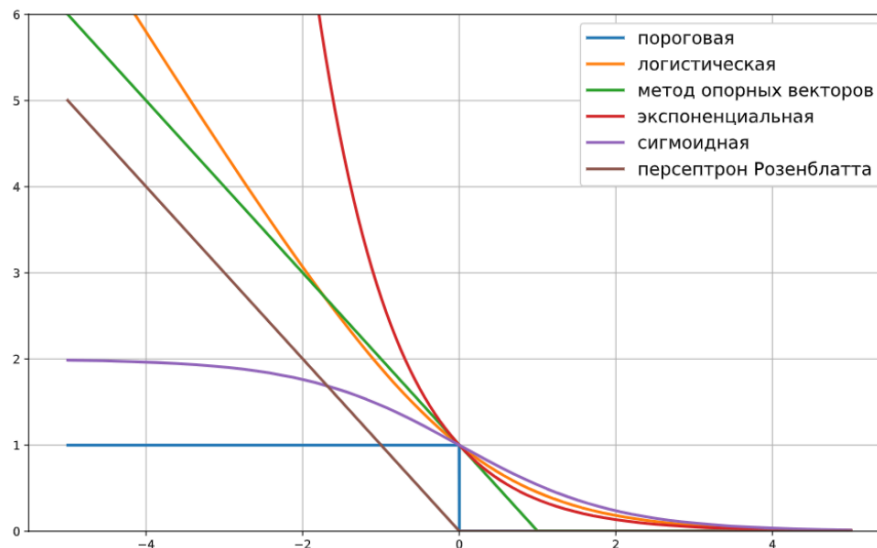
$$\frac{1}{l} \sum_{i=1}^l \left[\underbrace{\{w; x_i\} y_i}_M < 0 \right] \rightarrow \min_w$$

Что-то очень дискретное и очень грустно дифференцируемое, поэтому попробуем накинуть гладку верхнюю оценку, которую затем и будем минимизировать, надеясь что эта минимизация зажмёт и нашу основную

$$0 \leq L(M) \leq \bar{L}(M)$$

В целом, использование какой-либо верхней оценки зависит от постановки задачи, т.к. каждая аппроксимация приводит к своим свойствам. Приведём примеры двух таких верхних оценок, устройство которых подробно рассмотрим далее:

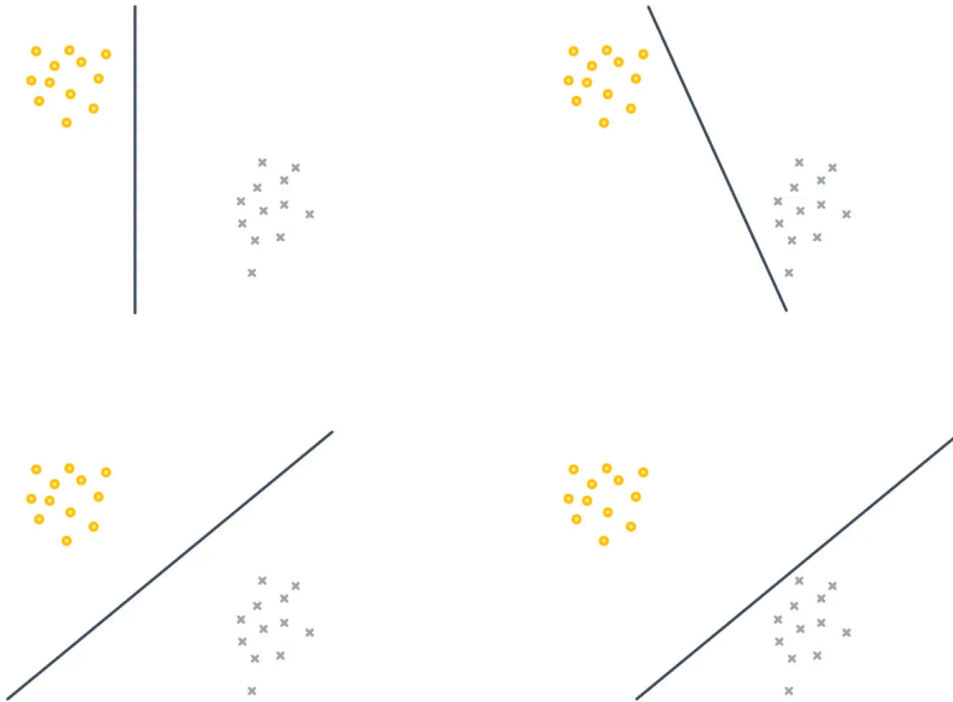
- (i) $\bar{L}(M) = \ln(1 + e^{-M})$ — логистическая функция потерь, позволяет оценивать вероятности
- (ii) $\bar{L}(M) = \max(0, 1 - M)$ — SVM, обычно лучше по метрикам качества



13. Что такое метод опорных векторов? Опишите его основную идею. На что влияет гиперпараметр C?

Идея SVM: в целом мы можем провести довольно много разделяющих поверхностей, которые будут как-то классифицировать объекты на классы. Но у нас может возникнуть логичное желание провести такую разделяющую

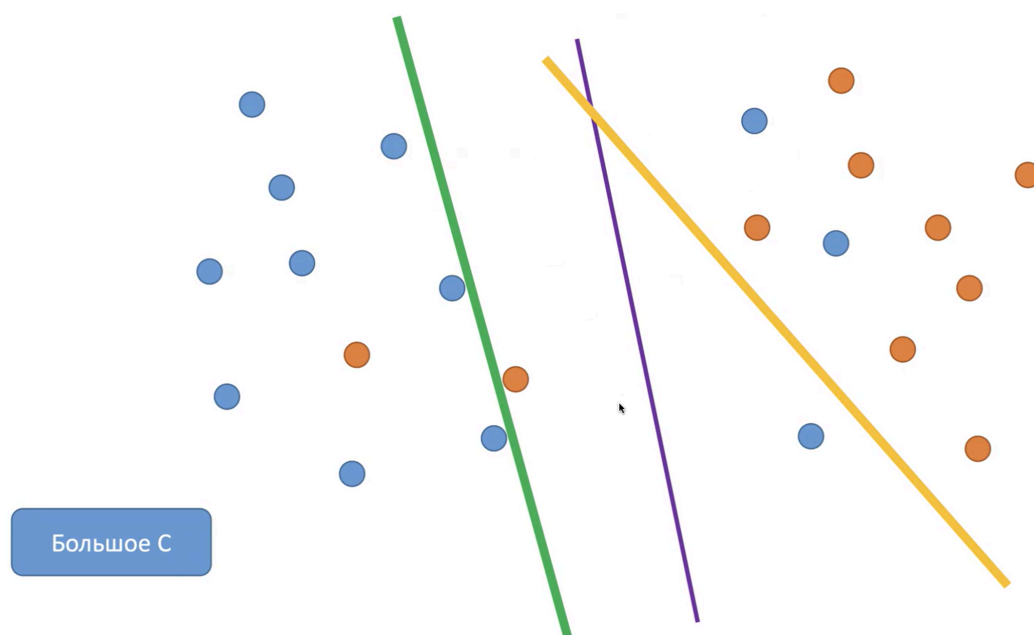
поверхность, которая будет находиться на максимально удаленном расстоянии от обоих классов, при этом с минимальным количеством ошибок.



Запишем задачу SVM:

$$\begin{cases} \frac{1}{\|w\|^2} \rightarrow \max_{w, w_0} \\ (y_i < w, x_i >) \geq 1 - \varepsilon_i \\ \varepsilon_i \geq 0 \end{cases} \rightarrow \begin{cases} \|w\|^2 + C \sum_{i=1}^l \varepsilon_i \rightarrow \min \\ (y_i < w, x_i >) \geq 1 - \varepsilon_i \\ \varepsilon_i \geq 0 \end{cases}$$

Гиперпараметр C отвечает за то, насколько сильно мы разрешаем модели ошибаться.



Объединим последние два условия:

$$\begin{cases} \varepsilon_i \geq 0 \\ \varepsilon_i \geq 1 - (y_i < w, x_i >) \end{cases} \rightarrow \varepsilon_i = \max \left(0, \underbrace{1 - (y_i < w, x_i >)}_{\text{один минус отступ}} \right)$$

Тогда получим итоговую верхнюю оценку, на который обучаемся:

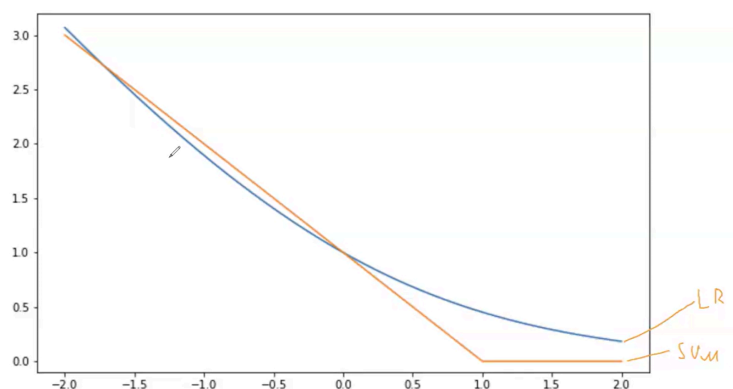
$$\underbrace{\|w\|^2}_{\text{L2 регуляризатор/штраф за малый отступ}} + \underbrace{C \sum_{i=1}^l \max(0, 1 - (y_i < w, x_i >))}_{\text{функционал ошибки}} \rightarrow \min_w$$

14. Чем отличаются функции потерь в логистической регрессии и в SVM?

Хммм, ну...почти что всем :)

Для логрега: $\frac{1}{l} \sum_{i=1}^l \log(1 + e^{-M}) \rightarrow \min$

Для SVM: $\|w\|^2 + C \sum_{i=1}^l \max(0, 1 - M) \rightarrow \min$



- SVM всё равно на отступ, когда он больше 1, там везде ошибка в ноль ударяется

- Для логистической - нет, она стимулирует отступ устремляться в $+\infty$, поэтому логрег хорошо оценивает вероятности
- SVM, скорее всего, будет лучше по метрикам качества

15. Как можно свести задачу многоклассовой классификации к серии задач бинарной классификации?

Идея - для каждого класса организуем модельку, которая будет отличать объекты только этого класса от всего остального, так называемый One-vs-rest.

Имеем:

- К классов: $\mathbb{Y} = \{1, \dots, K\}$
- $X_k = (x_i; [y_i] = k)_{i=1}^l$ - сконструировали для каждого класса его собственную выборку: объекты взяли все, но определили принадлежат ли ответы на этих объектах k-ому классу.
- обучаем $a(x)_k, k = 1, \dots, K$ на X_k
- моделька должна выдавать уверенности, т.е. оценки принадлежности объекта своему классу
- На вход поступает новый объект, тогда мы делаем прогноз как:

$$a(x) = \arg \max_{k=1, \dots, K} a(x)_k$$

16. Что такое микро- и макро-усреднение?

Что тут можно сказать? Ну просто во...

Микро-усреднение	Макро-усреднение
Вычисляем TP_k, FP_k, FN_k, TN_k для каждого класса	Вычисляем нужную метрику для каждого класса (например, $precision_1, \dots, precision_K$)
Суммируем по всем классам, получаем TP, FP, FN, TN	Усредняем по всем классам
Подставляем их в формулу для precision/recall/...	
Крупные классы вносят больший вклад	Игнорирует размеры классов